

Module 3 — Integration: ETL Pipeline

Build a Python ETL pipeline that extracts data from PostgreSQL, transforms it with Pandas, validates data quality, and loads the result to a new database table and CSV file.

Due: Day 4 (Wednesday) end of day. Resubmissions are accepted through Saturday of the assignment week.

Setup

- Accept the assignment:** [Accept the M3 Integration assignment](#)
- Clone your repo and open it in VS Code.
- Start your Postgres container (reuse from the lab, or create a new one):

```
docker run -d --name postgres-m3-int \
-e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres \
-e POSTGRES_DB=amman_market \
-p 5432:5432 -v pgdata_m3_int:/var/lib/postgresql/data \
postgres:15-alpine
```

- Create the database and load the schema + seed data:

```
psql -h localhost -U postgres -d amman_market -f schema.sql
psql -h localhost -U postgres -d amman_market -f seed_data.sql
```

- Verify: `psql -h localhost -U postgres -d amman_market -c "\dt"` should show 4 tables.
- Install dependencies: `pip install -r requirements.txt`. This installs `psycopg[binary]` and `sqlalchemy` (new packages for this module) along with `pandas` and `pytest`.

The Database

Amman Digital Market is a fictional online marketplace with four tables:

Table	Description	Rows	Key columns
customers	Registered users	~100	customer_id (PK), name, city, registration_date
products	Product catalog	~30	product_id (PK), name, category, unit_price
orders	Customer orders	~500	order_id (PK), customer_id (FK), order_date, status
order_items	Line items per order	~1200	item_id (PK), order_id (FK), product_id (FK), quantity

Data quality issues in the seed data (intentional):

- Some orders have `status = 'cancelled'` — these should be excluded from analytics
- Some customers have `NULL` city values
- Some `order_items` have `quantity > 100` — likely data entry errors; these should be flagged and excluded

Your Task

Complete the implementation of `etl_pipeline.py`. The file contains function signatures with docstrings — you implement the logic.

```
extract(engine)
```

Connect to PostgreSQL via SQLAlchemy. Extract all four tables into DataFrames. Return a dictionary:

```
{"customers": df, "products": df, "orders": df, "order_items": df}.
```

```
transform(data_dict)
```

- Join `orders` with `order_items` and `products` to get full order details
- Compute `line_total = quantity * unit_price` for each order item
- Filter out cancelled orders (`status = 'cancelled'`)
- Filter out suspicious quantities (`quantity > 100`)
- Aggregate to a customer-level summary with columns:
 - `customer_id`
 - `customer_name`
 - `total_orders` — count of distinct orders
 - `total_revenue` — sum of `line_total`
 - `avg_order_value` — `total_revenue / total_orders`
 - `top_category` — the product category with the highest total revenue for that customer

Return the customer summary DataFrame.

```
validate(df)
```

Run data quality checks on the transformed DataFrame:

- No nulls in `customer_id` or `customer_name`
- `total_revenue > 0` for all customers
- No duplicate `customer_id` values
- `total_orders > 0` for all customers

Print each check with PASS/FAIL status. Return a dictionary of check results. Raise `ValueError` if any critical check fails.

```
load(df, engine, csv_path)
```

- Write the customer summary DataFrame to a new `customer_analytics` table in PostgreSQL (`if_exists="replace"`)
- Save to `output/customer_analytics.csv`
- Print the row count loaded

```
main()
```

Orchestrate: `extract` → `transform` → `validate` → `load`. Print progress and row counts at each stage. Include the `if __name__ == "__main__":` guard.

Tests

Write at least 3 tests in `tests/test_etl.py`:

- `test_transform_filters_cancelled` — Create test DataFrames with a cancelled order. Confirm it's excluded after transform.
- `test_transform_filters_suspicious_quantity` — Create test DataFrames with quantity > 100. Confirm it's excluded.
- `test_validate_catches_nulls` — Create a DataFrame with null `customer_id`. Confirm `validate()` raises `ValueError`.

Run your tests: `pytest tests/test_etl.py -v`

README

Complete the `README.md` with:

- Overview:** What this pipeline does
- Setup:** How to start Postgres and load the data
- How to run:** `python etl_pipeline.py`
- Output:** What `customer_analytics.csv` contains
- Quality checks:** What validations are performed and why

Challenge Extensions

These are optional extensions for learners who complete the base requirements.

Tier 1 — Advanced Quality Checks and Reporting

- Add statistical outlier detection: flag customers whose `total_revenue` is more than 3 standard deviations from the mean. Add an `is_outlier` column to the summary.
- Generate a data quality report as `output/quality_report.json` with: total records checked, checks passed/failed, flagged outliers (customer IDs and values), timestamp of the ETL run.

Tier 2 — Incremental ETL with Change Detection

- Modify the ETL to support incremental loading: on subsequent runs, only process orders newer than the last ETL run timestamp.
- Create a `etl_metadata` table in Postgres to track run history (`run_id`, `start_time`, `end_time`, `rows_processed`, `status`).
- Compare full vs. incremental run: row counts and execution time. Document the tradeoffs in your README.

Tier 3 — ETL Framework with Configuration and Logging

- Build a reusable ETL framework: the pipeline reads a JSON config file specifying source tables, join keys, filters, aggregation rules, validation checks, and output targets.
- Replace `print()` statements with Python's `logging` module. Use structured log output with timestamps, log levels, and stage names.
- Demonstrate the framework on two different configs: the original e-commerce pipeline AND a second pipeline you design against a different subset of the data (e.g., product-level analytics instead of customer-level).
- Design goal: adding a new ETL pipeline requires only a new config file — no new Python code.

Submit

- Create branch `integration-3/etl-pipeline`
- Commit: `etl_pipeline.py`, `tests/test_etl.py`, `README.md`, `output/customer_analytics.csv`
- Open a PR to `main`
- Your PR description must include:
 - Row counts at each ETL stage (extracted, transformed, loaded)
 - Validation check results (PASS/FAIL for each check)
- Paste your PR URL into TalentLMS → Module 3 → Integration 3 to submit this assignment.

